

Solving Multiphysics and Multiscale Problems with AI-Augmented Simulations (CFD-PINN)

From Differential Equations to Physics-Informed Neural Networks, in One Session

Jeff Gomes and Henglai Zhan

(Twitter/X: @CoPhyMs)



XXIX ENMC
ENCONTRO NACIONAL DE MODELAGEM COMPUTACIONAL

XVII ECTM
ENCONTRO DE CIÊNCIA E TECNOLOGIA DE MATERIAIS

Hours of CFD, or milliseconds of inference?

The question for today

A high-fidelity CFD run can take days on a cluster. Can a neural network that *knows the physics* give the same answer in milliseconds?

Short answer: sometimes, and understanding *when* is the point of this lecture.



Why PINNs?

Motivation and a short history.



The language of physics

PDEs, boundary conditions, well-posedness.



CFD and its bottleneck

Why classical solvers are so expensive.



Anatomy of a PINN

Network + automatic differentiation + loss.



Will it work?

Error sources and training pathologies.



Making PINNs train

Activations, optimisers, formulations.



Real flows

Cylinder flow and CO₂ injection.

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion

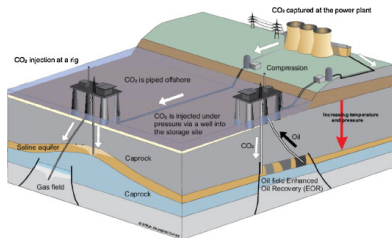
MOTIVATION

CMFD-PINN for Predicting CO₂ Flows in Underground Storage (2024-27)

Henglai Zhan, Zaibin Lin, Francisco Oliveira, Jeff Gomes

Main Aim

Develop a model to predict the onset and evolution/growth of viscous instabilities (expressed as **fingers**) at different conditions with the integration of traditional computational multi-fluid flow dynamics (CMFD) methods and physics-informed neural networks (PINNs) during geological CO₂ storage.



Schematic of CCS infrastructure and relationship with geological structure (extracted from [doi:10.1016/j.ijggc.2012.07.020](https://doi.org/10.1016/j.ijggc.2012.07.020)).

MOTIVATION

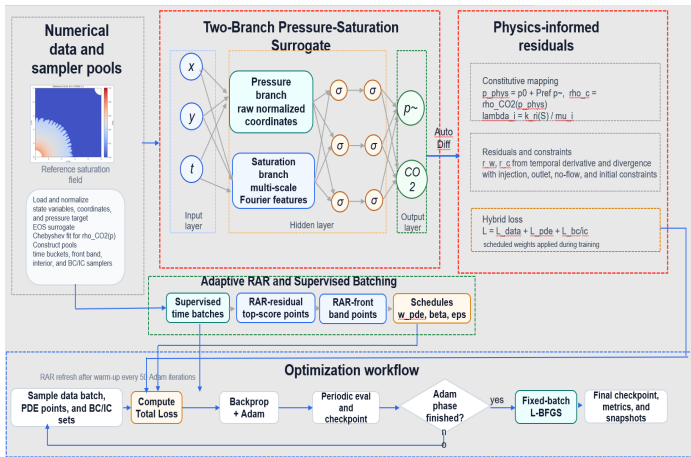
CMFD-PINN for Subsurface CO₂ Flows

Main Issues

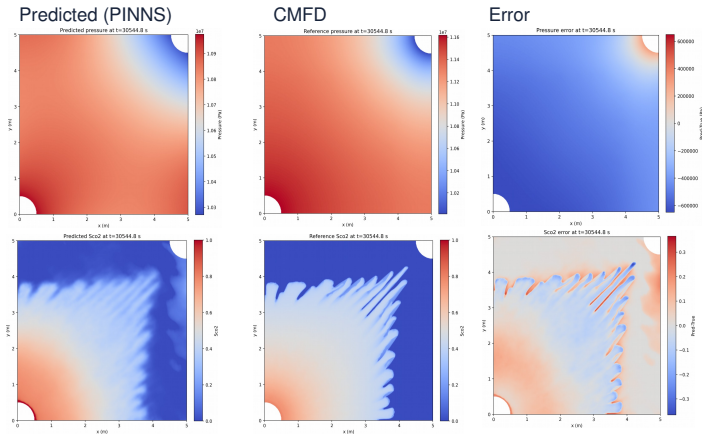
- Traditional CMFD: High computational cost for parametric sweeps.
- Vanilla PINNs: Struggle to resolve sharp, multiscale fingering fronts.

- 1 **Strategy:** High-order accurate data from numerical simulations coupled with PINN.
- 2 **Architecture Improvement:** A decoupled **pressure-saturation network with multi-scale Fourier features** is adopted to resolve the variable disparity between smooth pressure and oscillatory saturation.
- 3 **Training Strategy:** Front-weighted residual-based adaptive refinement (RAR) sampling is implemented, concentrating training effort on the fingers' tips and transition zones to minimize local geometric errors.

CMFD-PINN for Subsurface CO₂ Flows



CMFD-PINN for Subsurface CO_2 Flows



(Top: pressure; Bottom: CO_2 saturation) The model effectively reproduces the macroscopic propagation direction and fingering morphology. Also, the localised discrepancies observed around rapidly advancing tips and transition bands provide valuable insights for further refining front-aware architectures.

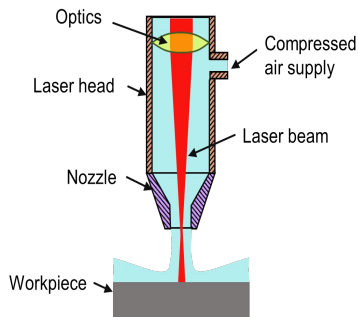
Laser Cutting for Decommissioning: O&G and Nuclear Waste (2022-25)

Chathura Ranasinghe, Ketan Ashara, Marcus Bannermann, Richard Neilson, Jeff Gomes

Knowledge Transfer Partnership (KTP)



Laser Cutting for Decommissioning: O&G and Nuclear Waste (2022-25)



MOTIVATION

Laser Cutting for Decommissioning: O&G and Nuclear Waste (2022-25)

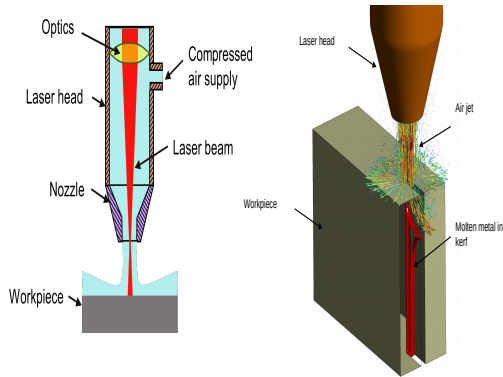
Multi-physics problem involving coupled

- 1 Multiphase CFD: design of the nozzle, super-/hyper-sonic flows, turbulence flow model, compressibility;
- 2 Solids: Phase change (melting, vaporisation and plasma)

Aim: Design a virtual underwater laser cut tool to assist decommissioning activities.

Challenge problem for traditional multiphysics and multiscale methods/models.

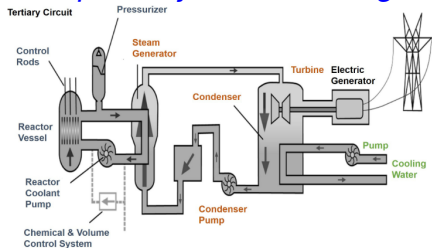
Suggestion: To couple high-order accurate self-adaptive CMFD methods with advanced predictive methods and ML algorithms !



3D FVM simulations with $\sim 1.2\text{M}$ cells in 40 cores (Maxwell Cluster) – 20ms / day.

Integrating Computational Multi-Fluid Dynamics (CMFD) and AI for Pressurised Water Reactors (2022-25)

Stephen Ajah, Evald Bregu, Lateef Akanji and Jeff Gomes



Research on:

1. Development of higher-order flux limiter and an adaptive unstructured mesh algorithm to capture the multifluid interface during post-CHF events in PWR.
2. Numerical investigation of critical heat flux (CHF) and core relocation to accurately predict accident progression.
3. Development of a novel ML-integrated higher-order model that combines control volume and finite element methods (CVFEM), a compressive advection interface capturing scheme (CAICM), and an ML-parameterized equation of state (ML/EoS), on flow dynamics during boiling transitions to CHF event stage in PWRs.
4. Development of a DL incorporated model from parameterised equation of state (EoS) to predict the likelihood of CHF occurrence, which normally precedes the LOCA and SA events in PWR, with automated extracted bubble dynamics datasets.

- What is the nature of the cooling flow associated with the core during severe accident events?
- Development of the model to accommodate **uncertainties, high T & P conditions and phase changes** associated with the system.

MOTIVATION

Integrating Computational Multi-Fluid Dynamics (CMFD) and AI for Pressurised Water Reactors (2022-25)

- ▶ 2D bubble volume fraction, temperature and mesh evolution

- ▶ 2D bubble volume fraction, temperature and mesh evolution for higher thermal condition

- ▶ 3D bubble volume fraction evolution

- ▶ 3D thermal boundary layer evolution

Integrating Computational Multi-Fluid Dynamics (CMFD) and AI for Pressurised Water Reactors (2022-25)

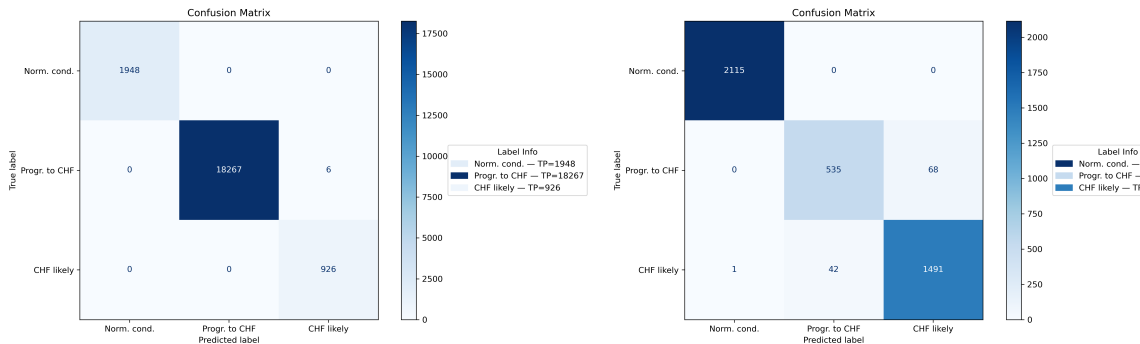


Figure: Confusion matrix of the CHF prediction: (a) for 2D simulation results and (b) for 3D simulation results. TP is the total class of datapoint captured for each predicted event.

Physics Informed Neural Networks: why?

- a Classical solvers (FEM, FVM, spectral methods) discretise space and time and march the physics forward, mesh cell by mesh cell.
- b PINNs instead encode the physics — the governing PDE, boundary and initial conditions — directly into the **loss function** of a neural network.
- c The network is trained to minimise the residual of the equations everywhere in the domain, not just at grid points.
- d Solving a physical problem becomes a hybrid of **physics** (the equations) and **data** (measurements, when available).

Takeaway

PINNs are an additional tool for solving (P)DEs — not a replacement for classical solvers.

They face difficulties classical solvers do not, but many can be tackled with insights from deep learning.

PINNs work best when they are actually *data-informed*.

Physics Informed Neural Networks: a historical note



1998 — Lagaris, Likas & Fotiadis

First explicit method to solve initial and boundary value problems using artificial neural networks.



2001 — Aarts & Van Der Veer

Solve PDEs and their boundary/initial conditions with single-hidden-layer feedforward networks.



2011 — Kumar & Yadav (survey)

A survey of MLP and radial-basis-function network methods for solving differential equations.



2017–2019 — Raissi, Perdikaris & Karniadakis

Coin “Physics-Informed Deep Learning”; formalise forward/inverse problems using automatic differentiation.

What changed after 2017 was not the idea but the tools: GPUs and automatic differentiation in PyTorch, TensorFlow and JAX.

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs**
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion

What is a differential equation?

- An equation relating an unknown function to its own derivatives.
- An **ODE** involves derivatives with respect to a single variable (*e.g.*, time); a **PDE** involves partial derivatives with respect to several variables (*e.g.*, space *and* time).
- Almost every conservation law in physics and engineering — heat, momentum, mass, charge — is naturally written as a PDE.
- Solving one means finding a **function**, not a number: the unknown lives in an infinite-dimensional space of functions.

Simplest example, exponential decay: $\frac{du}{dt} = -k u(t)$.

General form

$$\mathcal{N}_\lambda[u](x) = f(x), \quad x \in \Omega$$

$$\mathcal{B}[u](x) = g(x), \quad x \in \partial\Omega$$

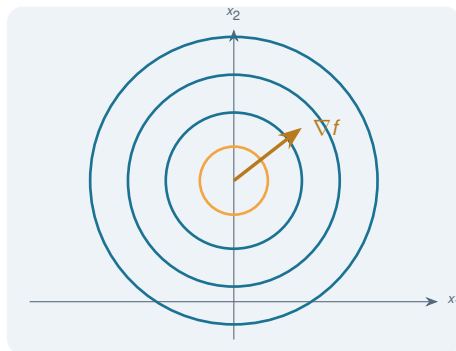
$\mathcal{N}_\lambda$: differential operator with physical constants λ (diffusivity, viscosity, ...);
 f : source; u : unknown field; $\mathcal{B}$: boundary/initial operator.

Equation + conditions = the problem every solver, classical or neural, actually solves.

The derivative machinery

- The derivative $f'(x)$ is the limit of the **difference quotient** $\frac{f(x+h) - f(x)}{h}$ as $h \rightarrow 0$.
- The **Taylor expansion** rebuilds f near a point from its derivatives there:

$$f(x+h) = \sum_n \frac{f^{(n)}(x)}{n!} h^n.$$
- For $f: \mathbb{R}^n \rightarrow \mathbb{R}$, the **partial derivative** $\partial f / \partial x_i$ freezes every other variable; stacking them gives the **gradient** ∇f .
- Second and mixed partials (u_{xx} , $\partial^2 u / \partial x \partial t$) appear constantly in PDEs.
- This is exactly the machinery **automatic differentiation** will later compute for us, exactly, inside a neural network.



Level sets of f ; ∇f is normal to them, pointing toward higher values

CLASSIFICATION

Three families of PDEs, three kinds of boundary condition

Elliptic

Laplace: $\nabla^2 u = 0$

Poisson: $\nabla^2 u = f$

Darcy flow, Helmholtz

Equilibrium: no time.

Parabolic

Heat: $u_t = \alpha \nabla^2 u$

Burgers, Fokker–Planck

Advection–diffusion

Diffusion: smoothing in time.

Hyperbolic

Wave: $u_{tt} = c^2 \nabla^2 u$

Advection/Transport

Propagation: fronts and waves.



Dirichlet

value of u prescribed on $\partial\Omega$



Neumann

normal gradient $\partial u / \partial n$ prescribed



Robin

linear combination of the two

Classification (2D, second order): elliptic $B^2 - 4AC < 0$, parabolic $= 0$, hyperbolic > 0 .

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD**
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion

Conservation Laws for Fluid Flow

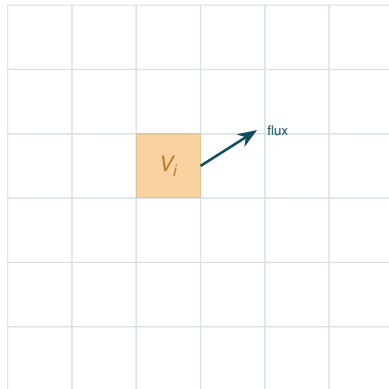
- Computational Fluid Dynamics (CFD) solves the equations that express conservation of **mass**, **momentum**, and **energy** for a moving fluid.
- These are the **Navier–Stokes equations (NSE)**, a coupled, nonlinear system of PDEs.
- The nonlinear advection term $(\mathbf{u} \cdot \nabla) \mathbf{u}$ is the source of most of CFD's difficulty and of turbulence itself.
- No general closed-form solution exists: we must compute.

Incompressible Navier-Stokes Equations

$$\begin{cases} \nabla \cdot \mathbf{u} = 0 \\ \rho \left[\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right] = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f} \end{cases}$$

Why is CFD so expensive?

- A computer stores finitely many numbers, so **discretisation** replaces the domain by a mesh (FDM, FVM, FEM) and the fields by values on it.
- Most engineering flows are **turbulent**. DNS resolves every scale, but grid points scale as $Re^{9/4}$ in 3D; RANS and LES model the unresolved scales instead, which raises the **closure problem**.
- A single high-fidelity run can take hours to weeks on HPC, and design, optimisation and UQ need *thousands* of runs, each possibly with a new mesh.
- **The PINN pitch:** once trained, a network evaluates the solution in milliseconds, anywhere in the domain.



A finite-volume mesh: conservation is enforced cell by cell

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN**
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion

MODEL PROBLEM

The 1+1-D heat equation

- Heat diffuses along a rod $x \in [0, L]$ with thermal diffusivity α ; both ends are held at zero temperature.

$$u_t = \alpha u_{xx}, \quad x \in (0, L), \quad t > 0$$

$$\text{BCs: } u(0, t) = u(L, t) = 0$$

$$\text{IC: } u(x, 0) = u_0(x)$$

- Simple enough to solve by hand, so we can measure a PINN's *true* error, not just its training loss.
- A PINN replaces the unknown u by a network $u_\theta(x, t)$ and turns each line of the problem into a loss term.

The loss terms

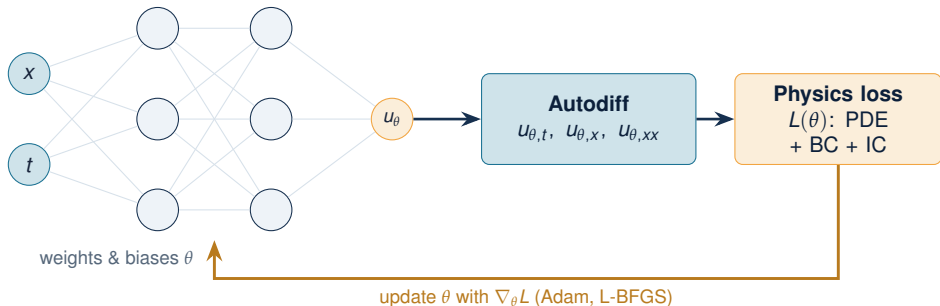
$$L_{\text{PDE}} = \frac{1}{N_f} \sum_i |u_{\theta,t} - \alpha u_{\theta,xx}|^2_{(x_i, t_i)}$$

$$L_{\text{BC}} = \frac{1}{N_b} \sum_i |u_\theta(0, t_i)|^2 + |u_\theta(L, t_i)|^2$$

$$L_{\text{IC}} = \frac{1}{N_0} \sum_i |u_\theta(x_i, 0) - u_0(x_i)|^2$$

$$L = \lambda_r L_{\text{PDE}} + \lambda_b L_{\text{BC}} + \lambda_0 L_{\text{IC}} (+ \lambda_d L_{\text{data}})$$

Automatic differentiation closes the loop



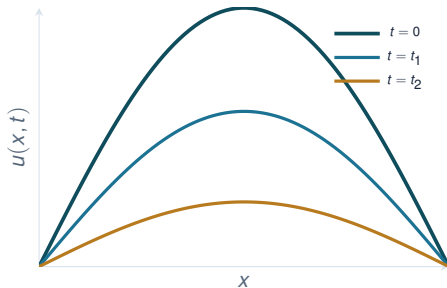
- The network is just a smooth function of (x, t) , so the chain rule gives its derivatives **exactly**: no mesh, no finite differences.
- The *same* autodiff engine computes $\nabla_\theta L$ for training: one tool, used twice.
- Collocation points can be sampled anywhere, which is why PINNs are **mesh-free**.

BENCHMARK

The exact answer we will compare against

- **Separation of variables:** seek $u(x, t) = X(x) T(t)$; the PDE splits into two ODEs.
- The Dirichlet BCs force X into sine modes $\sin(n\pi x/L)$: an eigenvalue problem, *i.e.*, projecting u_0 onto a Fourier sine basis.
- Each mode decays at a rate set by its frequency: **higher modes die faster**.

$$u(x, t) = \sum_{n=1}^{\infty} b_n e^{-\alpha(n\pi/L)^2 t} \sin\left(\frac{n\pi x}{L}\right)$$



Temperature profile draining out through the cold ends

EXAMPLE 1

Cup of Coffee (Newton's cooling law)



[Link to GitHub](#)

A hot cup of coffee at initial temperature T_o cools towards room temperature T_∞ , following Newton's law of cooling:

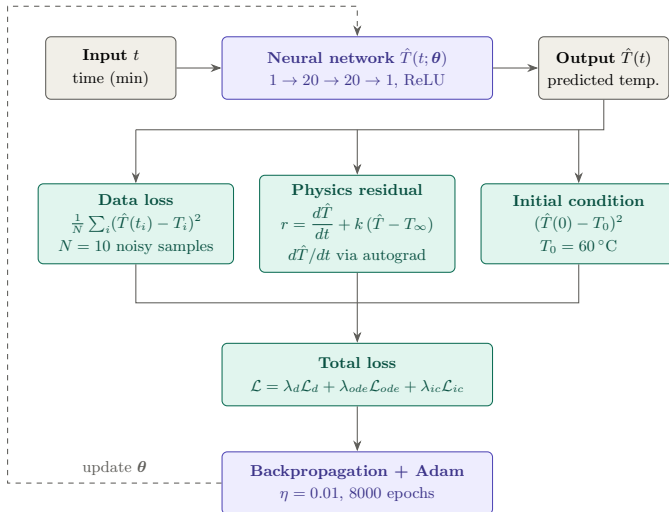
$$\frac{dT}{dt} = -\frac{hA_s}{mC_p} (T - T_\infty),$$

where h , A_s and (mC_p) are the heat transfer coefficient, surface area and effective heat capacity. The analytical solution is

$$T(t) = T_\infty + (T_o - T_\infty) e^{-t/\tau}, \quad \tau = \frac{mC_p}{hA_s}.$$

Try it: scan the QR code and run the notebook. Seven steps: exact solution $\rightarrow$ network $\rightarrow$ residual via autograd $\rightarrow$ BC/IC $\rightarrow$ assemble $\rightarrow$ train $\rightarrow$ test against the exact solution.

Analytical Solution + PINN



Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs**
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion

PAUSE & THINK

Your loss is 10^{-6} . Is your solution right?

Discuss with your neighbour (1 minute)

You trained a PINN and the loss dropped to 10^{-6} . List every reason the network could *still* be far from the true solution.

Hints: the loss was evaluated only at the points you sampled. Was the network big enough? Did the optimiser find the best weights, or just *some* weights?

Three sources of error, three levers

Optimisation error

Gap between the trained weights and the best possible weights: the loss is nonconvex.

Lever: optimiser

Generalisation error

Gap between the loss on sampled points (empirical risk) and over the whole domain (true risk).

Lever: sampling

Approximation error

How well the chosen architecture can represent the true solution at all.

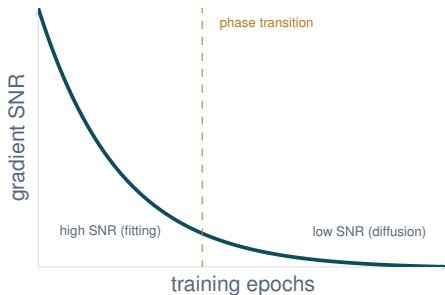
Lever: architecture

$$\|u_\theta - u\| \lesssim \varepsilon_{\text{opt}} + \varepsilon_{\text{gen}} + \varepsilon_{\text{approx}} \quad (\text{Kutyniok, 2022})$$

De Ryck, Lanthaler & Mishra (2021): two hidden tanh layers suffice to make $\varepsilon_{\text{approx}}$ small. De Ryck & Mishra (2021): for Kolmogorov-type PDEs (heat, Black–Scholes), ε_{gen} is **dimension-independent**, which is why PINNs shine in high dimensions. **But a good network existing does not mean training will find it.**

When the loss terms pull in different directions

- **Information Bottleneck** view of training: a fast **fitting** phase (high gradient SNR, consistent directions) is followed by a slow **diffusion** phase (low SNR) where most generalisation happens.
- [Anagnostopoulos et al. \(2024\)](#): in PINNs, the PDE, BC and IC terms each have their own gradient.
 - **Aligned** gradients $\Rightarrow$ fitting-dominated.
 - **Conflicting** gradients $\Rightarrow$ noise-dominated; training stalls.
- Remedies: re-weight the λ 's adaptively, sample smarter, or change the optimiser.



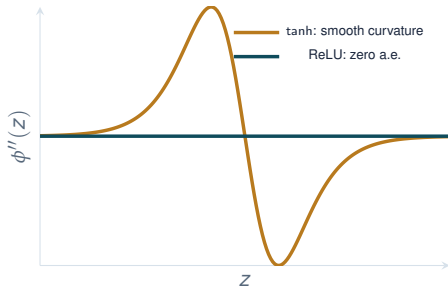
After [Schwartz-Ziv & Tishby \(2017\)](#).

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train**
- 7 From Toy Problems to Real Flows
- 8 Conclusion

Why does tanh dominate PINN architectures?

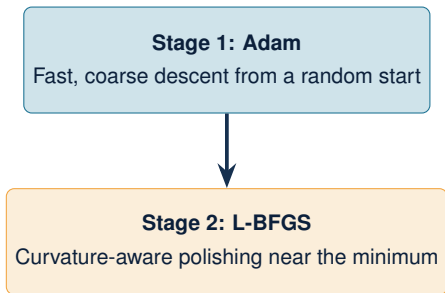
- **Standard DL (ReLU):** cheap and avoids vanishing gradients, but piecewise linear.
- **PINN requirement:** the residual needs high-order input derivatives (u_{xx} , u_{tttt} , ...).
- **The ReLU bottleneck:** its second derivative is zero almost everywhere, so $u_{\theta,xx}$ carries no curvature information.
- **Smooth alternatives** (tanh, sin, GELU) have well-behaved derivatives of every order.
- **Adaptive activation** (Jagtap et al., 2020): $\sigma(z) = \tanh(\alpha z)$ with trainable α learns the slope and accelerates convergence at negligible cost.



OPTIMISATION

First-order speed, second-order precision

- **SGD/Adam** use only the gradient: cheap and robust, but PINN loss landscapes are **ill-conditioned** (narrow, curved valleys), so they stall or oscillate near a minimum.
- **Newton's method** uses the inverse Hessian H^{-1} , far too expensive to form for a network.
- **BFGS** builds an approximation of H^{-1} from successive gradients; **L-BFGS** keeps only the last m (5–20) update pairs, so it scales to millions of parameters.



Close to a de-facto standard: Adam finds the right basin, L-BFGS polishes the solution once there.

Three ways to pose the loss: DGM vs. DRM vs. vPINN

Classical FEM picks a finite-dimensional trial space (piecewise polynomials on a mesh). The neural versions simply swap that space for a network u_θ , and differ in *which functional* they minimise.

DGM / PINN

Deep Galerkin: strong-form residual $\mathbb{E}_x |\mathcal{N}[u_\theta] - f|^2$, Monte Carlo collocation, no test space.

Sirignano & Spiliopoulos (2018).

Deep Ritz

Minimise an energy functional $\mathcal{E}[u_\theta]$; the weak form appears as its Euler–Lagrange equation. Lower-order derivatives.

E & Yu (2018).

Variational PINN

Petrov–Galerkin: choose a test basis (e.g., polynomials) and minimise the weak residual against it.

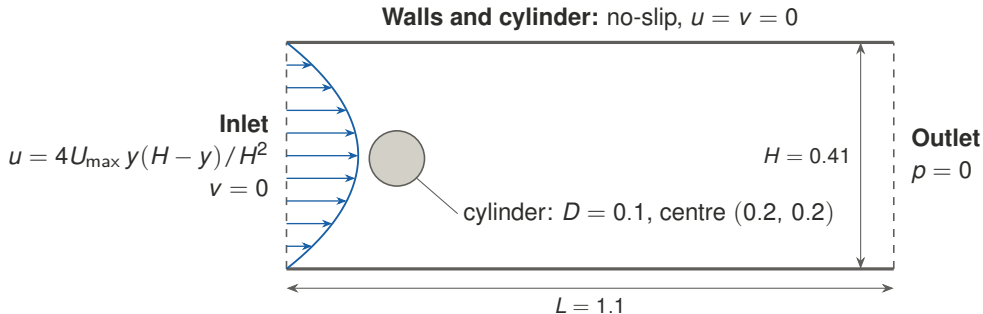
Kharazmi, Zhang & Karniadakis (2019).

Deep Ritz suits problems that come from energy minimisation (elasticity, potential flow); vPINN is closest in spirit to classical FEM.

Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows**
- 8 Conclusion

Example 2: Flow past a cylinder (CFD problem)



Steady incompressible Navier–Stokes

$$\nabla \cdot \mathbf{u} = 0$$

$$\rho (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \mu \nabla^2 \mathbf{u}$$

Unknowns: $\mathbf{u} = (u, v)$ and p

Parameters

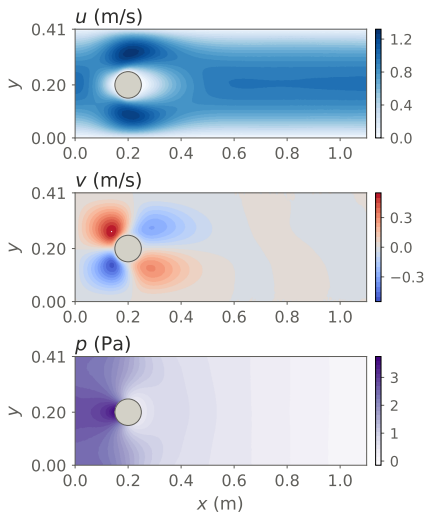
$$\rho = 1, \quad \mu = 0.02, \quad U_{\max} = 1, \quad D = 0.1$$

$$Re = \frac{\rho \bar{U} D}{\mu} \approx 3.3, \quad \bar{U} = \frac{2}{3} U_{\max}$$

Steady and laminar, far below vortex shedding

Geometry: DFG 2D-1 benchmark (Schäfer & Turek, 1996), shortened to $L = 1.1$; set-up after Rao, Sun & Liu (2020).

Example 2: CFD reference (ANSYS Fluent) vs. PINN



- 19,340 nodal values of u , v , p
- Gap acceleration: $u_{\max} = 1.3 U_{\max}$
- Stagnation on the upstream face:
 $p_{\max} = 3.63$; $p \rightarrow 0$ at the outlet
- $u \geq 0$ everywhere: no recirculation at $Re \approx 3.3$

Same physics, two routes

CFD: mesh + discretised equations

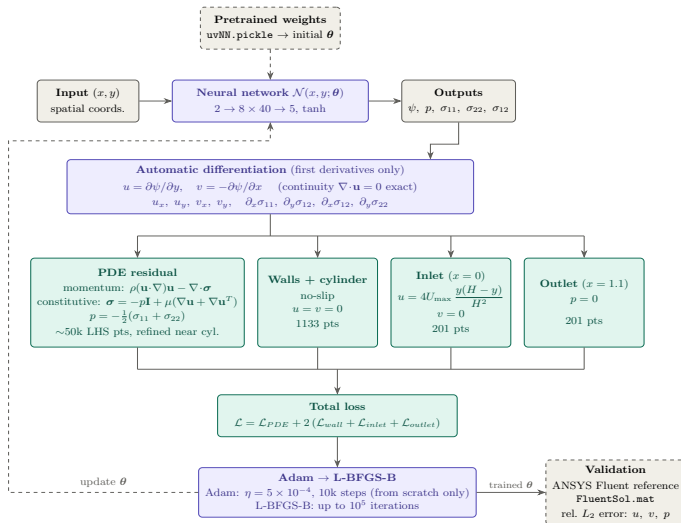
PINN: collocation points + residuals

Score: relative L_2 error of u , v , p at the Fluent nodes.

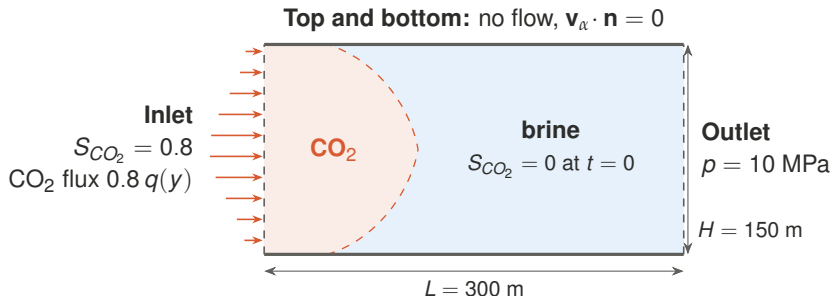


BLOCK DIAGRAM

CFD + PINN



Example 3: CO₂ Displacing Brine



Two-phase Darcy flow, $\alpha \in \{b, c\}$

$$\phi \partial_t S_\alpha + \nabla \cdot \mathbf{v}_\alpha = 0, \quad S_b + S_c = 1$$

$$\mathbf{v}_\alpha = -\frac{K k_{r\alpha}}{\mu_\alpha} \nabla p$$

Corey: $k_{rc} = \hat{S}^2$, $k_{rb} = (1 - \hat{S})^2$,
 with $\hat{S} = S_c/0.8$

Parameters

$$\phi = 0.2, \quad K = 10^{-12} \text{ m}^2, \quad t \in [0, 300] \text{ days}$$

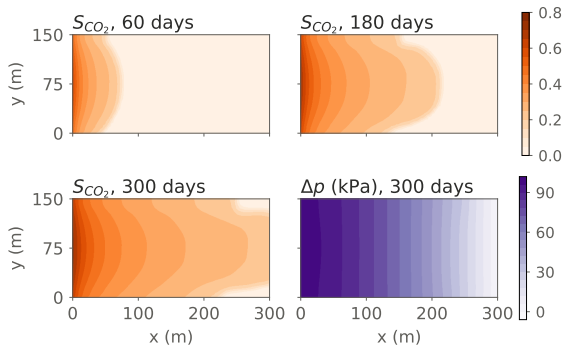
$$\mu_b = 10^{-3}, \quad \mu_c = 7 \times 10^{-5} \text{ Pa s}$$

$$\text{Mobility ratio } M = \mu_b / \mu_c \approx 14 \text{ (adverse)}$$

$$q(y) = q_0 \left[1 + \frac{1}{2} \cos 2\pi \left(\frac{y}{H} - \frac{1}{2} \right) \right], \quad q_0 = 10^{-6} \text{ m/s}$$

Neglected: gravity, capillarity, dissolution, compressibility, heterogeneity (teaching benchmark).

Example 3: Reference solution (ICFERST) and the PINN task



;

- 61×31 -vertex triangular mesh, $\Delta t = 1$ day, 16 snapshots
- Curved front: inflow $1.5 q_0$ at mid-height vs $0.5 q_0$ at the walls; breakthrough ≈ 260 days
- Mean S_{CO_2} tracks the injected volume ($+0.023$ per 20 days) until breakthrough
- Δp falls $233 \rightarrow 100$ kPa as mobile CO₂ replaces brine

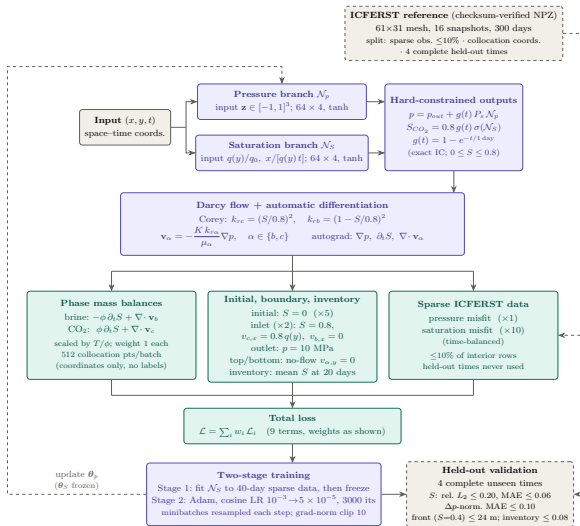
Role in the PINN study: data-informed

Train: sparse labels ($\leq 10\%$ of rows)

Physics: unlabelled collocation points

Score: unseen 20, 100, 200, 300 d

Reservoir Simulator (Porous Media CFD) + PINN



Section Outline

- 1 History & Motivation for PINNs
- 2 The Language of Physics – PDEs
- 3 Introduction to CFD
- 4 Anatomy of a PINN
- 5 Will It Work? Learning Theory of PINNs
- 6 Making PINNs Train
- 7 From Toy Problems to Real Flows
- 8 Conclusion**

SUMMARY

Key takeaways

- **The idea:** a PINN embeds the PDE residual, computed by automatic differentiation, directly into a network's loss, turning a universal function approximator into a mesh-free differential-equation solver.
- **The caveat:** a small loss is not a small error. Optimisation, generalisation and approximation errors, plus conflicting gradients between loss terms, explain *why* vanilla training can fail.
- **The levers:** smooth activations, Adam → L-BFGS, loss weighting and the choice of formulation (strong, energy, variational) turn a fragile prototype into a trainable model.
- **The big picture:** PINNs sit at the intersection of functional analysis, numerical PDEs and deep learning: multiphysics and multiscale simulation *augmented* by, not replaced by, AI.

PAUSE & THINK

CFD, PINN, or both? You decide

Scenario A

One 3D turbulent simulation of a new car body, needed once, at high accuracy.

Scenario B

Pricing an option on 100 correlated assets: a 100-dimensional Black–Scholes PDE.

Scenario C

A CO₂ store with pressure data from a few wells and uncertain permeability to infer.

Which tool would you reach for, and why? Use today's three ideas: cost of CFD, the curse of dimensionality, and PINNs being at their best when data-informed.